

## 06.1: MongoDB



UMD DATA605 - Big Data Systems

### 06.1: MongoDB

**Instructor:** Dr. GP Saggese, [gsaggese@umd.edu](mailto:gsaggese@umd.edu)

– @References@:

- All concepts in slides
- MongoDB tutorial
- Web
  - <https://www.mongodb.com/>
  - Official docs
- Seven Databases in Seven Weeks, 2e

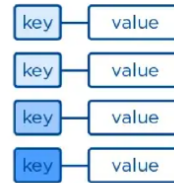


## 2 / 16: Key-Value Store vs Document DBs

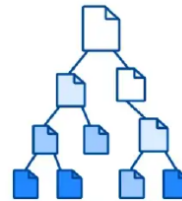
## Key-Value Store vs Document DBs

– **Key-value stores**

- Function as a map or dictionary
  - Examples: HBase, Redis
- Primarily retrieve values using keys
- Occasionally search within value fields using patterns
- Store uninterpreted values (e.g., binary blobs) linked to keys
- Use a single namespace for all key-value pairs

**Key-Value**– **Document DBs**

- Group key-value pairs into *documents*
  - Examples: MongoDB, CouchDB
- Documents formatted in JSON, XML, or BSON (binary JSON)
- Documents are part of *collections*
  - Comparable to *tables* in relational databases
- Large collections can be partitioned and indexed

**Document**

2 / 16

• **Key-Value Stores**

- These databases function like a map or dictionary, where each key is associated with a specific value.
  - \* *Examples include HBase and Redis.*
- They are optimized for retrieving values using keys, making them very fast for lookups.
- Occasionally, they allow searching within value fields using patterns, but this is not their primary function.
- Values are stored as uninterpreted data, such as binary blobs, which means the database does not understand the content of the values.
- All key-value pairs exist within a single namespace, simplifying the structure but limiting complex queries.

• **Document DBs**

- In these databases, key-value pairs are grouped into *documents*, which provide more structure.
  - \* *Examples include MongoDB and CouchDB.*
- Documents are formatted in JSON, XML, or BSON, allowing for more complex data representation.
- Documents are organized into *collections*, similar to tables in relational databases, which helps in managing related data.
- Large collections can be partitioned and indexed, improving performance for large datasets and complex queries.

## 3 / 16: MongoDB

## MongoDB

- **Developed by MongoDB Inc**
  - Founded in 2007
  - Based on DoubleClick's experience with large-scale data
  - Mongo comes from "hu-mongo-us"
- **Highly popular NoSQL database**
- **Document-oriented NoSQL DB**
  - Schema-less
    - No Data Definition Language (DDL) like SQL
    - Stores maps with any keys and values
    - Application manages schema, linking documents to meanings
  - Keys are string-stored hashes
    - Each document has a unique `_id` (reserved by MongoDB)
  - Values in BSON format
    - Based on JSON (B for Binary)
- **High-performance**
  - Developed in C++
  - Supports APIs (drivers) in various languages
    - Examples: JavaScript, Python, Ruby, Java, Scala, C++, etc



3 / 16

- **Developed by MongoDB Inc**
  - Founded in 2007, MongoDB Inc. was established to address the challenges of handling large-scale data, drawing from the experiences of DoubleClick, a company known for its advertising services.
  - The name "Mongo" is derived from the word "humongous," reflecting its capability to manage vast amounts of data.
- **Highly popular NoSQL database**
  - **Document-oriented NoSQL DB**
    - \* MongoDB is schema-less, meaning it doesn't require a predefined structure like traditional SQL databases. This flexibility allows developers to store data in a way that best suits their applications.
    - \* Instead of using a Data Definition Language (DDL), MongoDB stores data as maps with any keys and values, and the application itself manages the schema.
    - \* Each document in MongoDB has a unique identifier, `_id`, which is automatically reserved by the database.
    - \* Data is stored in BSON format, which is a binary representation of JSON, allowing for efficient data storage and retrieval.
- **High-performance**
  - MongoDB is developed in C++, which contributes to its high performance and efficiency.
  - It supports APIs (drivers) in various programming languages, making it versatile and accessible for developers using languages like JavaScript, Python, Ruby, Java, Scala, and C++. This wide range of support enhances its usability across different platforms and applications.

## 4 / 16: MongoDB: Example of Document

### MongoDB: Example of Document

---

- **A document is a JSON data structure**
- It corresponds to **a row in a relational DB**
  - Without schema
  - Primary key is `_id`
  - Values can be nested to an arbitrary depth

```
{
  "_id" : ObjectId("4d0b6da3bb30773266f39fea"),
  "country" : {
    "$ref" : "countries",
    "$id" : ObjectId("4d0e6074deb8995216a8309e")
  },
  "famous_for" : [
    "beer",
    "food"
  ],
  "last_census" : "Sun Jan 07 2018 00:00:00 GMT -0700 (PDT)",
  "mayor" : {
    "name" : "Ted Wheeler",
    "party" : "D"
  },
  "name" : "Portland",
  "population" : 582000,
  "state" : "OR"
}
```

4 / 16

- **MongoDB: Example of Document**
  - A document in MongoDB is structured as a *JSON data structure*. This means it uses a format that is easy to read and write for humans and machines. JSON stands for JavaScript Object Notation and is widely used for data interchange.
- **Corresponds to a Row in a Relational DB**
  - In traditional relational databases, data is stored in tables with rows and columns. In MongoDB, a document is similar to a row in these tables.
  - **Without Schema:** Unlike relational databases, MongoDB does not require a predefined schema. This allows for flexibility as each document can have different fields.
  - **Primary Key is `_id`:** Each document has a unique identifier called `_id`. This acts like a primary key in relational databases, ensuring each document can be uniquely identified.
  - **Values can be Nested:** MongoDB allows for complex data structures by supporting nested documents. This means values can be arrays or other documents, allowing for a rich representation of data.

## 5 / 16: MongoDB: Functionalities

## MongoDB: Functionalities

- **Design goals**
  - Performance
  - Availability/scalability
  - Rich data storage (not rich querying!)
- **Dynamic schema**
  - No DDL
  - Secondary indexes
  - Query language via API
- **Several levels of data consistency**
  - Atomic writes and fully-consistent reads (document level)
- **No joins nor transactions across multiple documents**
  - Distributed queries easy and fast
- **High availability through replica sets**
  - Primary replication with automated failover
- **Built-in sharding**
  - Horizontal scaling via automated range-based partitioning
  - Reads and writes distributed over shards

5 / 16

- **MongoDB: Functionalities**
  - **Design goals**
    - \* *Performance*: MongoDB is designed to be fast and efficient, making it suitable for applications that require quick data access and processing.
    - \* **Availability/scalability**: It is built to handle large amounts of data and users, ensuring that the system remains available and responsive even as demand grows.
    - \* **Rich data storage (not rich querying!)**: While MongoDB excels at storing complex data structures, it does not focus on providing advanced querying capabilities like some other databases.
- **Dynamic schema**
  - *No DDL*: MongoDB does not require a fixed schema, allowing for flexibility in how data is stored and modified without needing to define a structure beforehand.
  - **Secondary indexes**: These are used to improve query performance by allowing quick access to data based on non-primary key fields.
  - **Query language via API**: MongoDB provides a way to interact with the database using a query language through its API, making it accessible for developers.
- **Several levels of data consistency**
  - *Atomic writes and fully-consistent reads (document level)*: MongoDB ensures that operations on a single document are atomic, meaning they are completed fully or not at all, and reads are consistent at the document level.
- **No joins nor transactions across multiple documents**
  - *Distributed queries easy and fast*: By avoiding complex operations like joins and multi-document transactions, MongoDB can perform distributed queries more efficiently, which is beneficial for scalability.

- **High availability through replica sets**
  - *Primary replication with automated failover*: MongoDB uses replica sets to ensure data is replicated across multiple servers, providing high availability and automatic failover in case of server failure.
- **Built-in sharding**
  - *Horizontal scaling via automated range-based partitioning*: Sharding allows MongoDB to distribute data across multiple servers, enabling horizontal scaling by partitioning data into ranges.
  - **Reads and writes distributed over shards**: This ensures that both read and write operations can be spread across different shards, improving performance and scalability.

## 6 / 16: MongoDB: Hierarchical Objects

### MongoDB: Hierarchical Objects

- A Mongo **instance** has:
  - Zero or more “databases”
  - Mongo instance ~ Postgres instance
- A Mongo **database** has:
  - Zero or more “collections”
  - Mongo collection ~ Postgres tables
  - Mongo database ~ Postgres database
- A Mongo **collection** has:
  - Zero or more “documents”
  - Mongo document ~ Postgres rows
- A Mongo **document** has:
  - One or more “fields”
  - Always has a primary key `_id`
  - Mongo field ~ Postgres columns

```

{
  name: "al",
  age: 18,
  status: "D",
  groups: [ "politics", "news" ]
}

```

6 / 16

- **Mongo Instance**
  - A MongoDB instance can contain multiple databases, similar to how a Postgres instance operates. This is the top level of the hierarchy in MongoDB.
- **Mongo Database**
  - Each database within an instance can have multiple collections. This is analogous to a Postgres database, which contains tables.
- **Mongo Collection**
  - Collections are similar to tables in relational databases like Postgres. They hold multiple documents, which are akin to rows in a table.
- **Mongo Document**
  - Documents are the basic unit of data in MongoDB, similar to rows in Postgres. Each document contains fields, which are comparable to columns in a table. Every document has a unique primary key, `_id`, which ensures each document can be uniquely identified.

- **Fields**

- Fields within a document store the actual data and are similar to columns in a relational database. They can hold various data types and structures, allowing for flexible data modeling.

## 7 / 16: Relational DBs vs MongoDB: Concepts

### Relational DBs vs MongoDB: Concepts

| @RDBMS Concept@         | @MongoDB Concept@  | @Meaning in MongoDB@      |
|-------------------------|--------------------|---------------------------|
| database                | database           | Container for collections |
| relation / table / view | collection         | Group of documents        |
| row / instance          | document           | Group of fields           |
| column / attribute      | field              | A name-value pair         |
| index                   | index              | Automatic                 |
| primary keys            | <u>id</u> field    | Always the primary key    |
| foreign key             | reference          | Pointers                  |
| table joins             | embedded documents | Nested name-value pairs   |

```
{
  "_id" : ObjectId("4d0b6da3bb30773266f39fea"),
  "country" : {
    "$ref" : "countries",
    "$id" : ObjectId("4d0e6074deb8995216a8309e")
  },
  "famous_for" : [
    "beer",
    "food"
  ],
  "last_census" : "Sun Jan 07 2018 00:00:00 GMT -0700 (PDT)",
  "mayor" : {
    "name" : "Ted Wheeler",
    "party" : "D"
  },
  "name" : "Portland",
  "population" : 582000,
  "state" : "OR"
}
```

7 / 16

- **Relational DBs vs MongoDB: Concepts**

- **Database vs Database**

- In both relational databases (RDBMS) and MongoDB, a *database* serves as a container. In RDBMS, it holds tables, while in MongoDB, it contains collections.

- **Relation/Table/View vs Collection**

- A *collection* in MongoDB is similar to a table in RDBMS. It groups documents, which are akin to rows in a table.

- **Row/Instance vs Document**

- A *document* in MongoDB is a set of fields, similar to a row in a relational database. Each document can have a unique structure.

- **Column/Attribute vs Field**

- A *field* in MongoDB is a name-value pair, comparable to a column in RDBMS.

- **Index vs Index**

- Indexing in MongoDB is automatic, helping to speed up queries, similar to indexing in relational databases.
- **Primary Keys vs `_id` Field**
  - The `_id` field in MongoDB is always the primary key, ensuring each document is uniquely identifiable.
- **Foreign Key vs Reference**
  - *References* in MongoDB act like foreign keys, pointing to documents in other collections.
- **Table Joins vs Embedded Documents**
  - Instead of table joins, MongoDB uses *embedded documents*, allowing nested structures within a document for related data.
- **Example Document Explanation**
  - The JSON-like structure shown is a MongoDB document.
  - `_id`: Unique identifier for the document.
  - `country`: Uses a reference to another collection, similar to a foreign key.
  - `famous_for`: An array of strings, showing flexibility in data types.
  - `last_census`: Stores a date as a string.
  - `mayor`: An embedded document, demonstrating nested data.
  - `name`, `population`, `state`: Simple fields with straightforward data types.

## 8 / 16: Relational vs Document DB: Workflows

### Relational vs Document DB: Workflows

---

- **Relational DBs**
  - E.g., PostgreSQL
  - Know what to store
    - Tabular data
  - Static schema allows query flexibility (e.g., joins)
  - Complexity at insertion time
    - Decide data representation (schema)
- **Document DBs**
  - E.g., MongoDB
  - No assumptions on storage
    - E.g., irregular JSON data
  - Access data by key
    - Nested key-value map
  - Complexity at access time
    - Retrieve data from server
    - Process data client-side

8 / 16

- **Relational DBs**

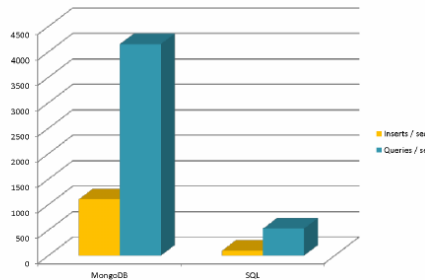


- *E.g., PostgreSQL*: This is a popular example of a relational database, which organizes data into tables.
- **Know what to store**: Relational databases are best for *tabular data*, meaning data that fits neatly into rows and columns.
- **Static schema allows query flexibility**: A *static schema* means the structure of the database is defined before data is entered. This allows for complex queries, like *joins*, which combine data from different tables.
- **Complexity at insertion time**: When adding data, you must decide how it fits into the predefined *schema*. This can be complex because you need to plan how data will be structured and related.
- **Document DBs**
  - *E.g., MongoDB*: This is a well-known document database, which stores data in a flexible, JSON-like format.
  - **No assumptions on storage**: Document databases can handle *irregular JSON data*, meaning data that doesn't fit neatly into tables.
  - **Access data by key**: Data is stored in a *nested key-value map*, which means you retrieve data using keys, similar to how you would access values in a dictionary.
  - **Complexity at access time**: The challenge here is when you *retrieve data from the server*. You often need to *process data client-side*, which can be complex if the data structure is irregular or deeply nested.

## 9 / 16: Why Use MongoDB?

### Why Use MongoDB?

- **Simple and powerful to query**
- **Fast**
  - 2-10x faster than PostgreSQL
- **Data model suitable for most web applications**
  - Semi-structured data
  - Quickly evolving systems
- Not suited for heavy, complex transaction systems
  - E.g., banking systems



9 / 16

- **Simple and powerful to query**
  - MongoDB offers a straightforward querying process, making it easy for developers to interact with the database. This simplicity is beneficial for rapid development and

iteration.

- **Fast**
  - MongoDB is noted to be 2-10 times faster than PostgreSQL. This speed advantage is crucial for applications requiring quick data retrieval and processing, enhancing user experience and system performance.
- **Data model suitable for most web applications**
  - *Semi-structured data*: MongoDB's flexible schema allows for storing semi-structured data, which is common in web applications where data formats can vary.
  - *Quickly evolving systems*: The adaptability of MongoDB's data model supports systems that need to evolve rapidly, accommodating changes without significant restructuring.
- **Not suited for heavy, complex transaction systems**
  - While MongoDB excels in speed and flexibility, it may not be ideal for systems requiring complex transactions, such as banking systems, where data integrity and transaction management are critical.

The accompanying chart visually demonstrates MongoDB's superior performance in terms of inserts and queries per second compared to SQL databases, highlighting its efficiency in handling large volumes of operations.

## 10 / 16: MongoDB: Data Model

### MongoDB: Data Model

- **Documents** are field-value pairs
  - *Field names*: strings
  - *Values*: any BSON type
    - Arrays of documents
    - Native data types
    - Other documents

```
{
  name: "sue",
  age: 26,
  status: "A",
  groups: [ "news", "sports" ]
}
```

← field: value  
← field: value  
← field: value  
← field: value

- **Examples:**
  - `_id`: ObjectId
  - `name`: document with fields `first` and `last`
  - `birth` and `death`: date type
  - `contribs`: array of strings
  - `views`: NumberLong

```
{
  _id: ObjectId("5099803df3f4948bd2f98391"),
  name: { first: "Alan", last: "Turing" },
  birth: new Date('Jun 23, 1912'),
  death: new Date('Jun 07, 1954'),
  contribs: [ "Turing machine", "Turing test", "Turingery" ],
  views: NumberLong(1250000)
}
```

10 / 16

- **Documents as Field-Value Pairs:**
  - In MongoDB, data is stored in *documents*, which are composed of field-value pairs.
  - **Field Names**: These are strings that act as keys in the document.
  - **Values**: Can be of any BSON type, which includes:
    - \* Arrays of documents: Allows for nested data structures.
    - \* Native data types: Such as strings, numbers, and booleans.

\* Other documents: Enables embedding documents within documents for complex data models.

- **Examples:**

- **\_\_id**: This is a unique identifier for each document, typically an `ObjectId`.
- **name**: A document containing fields like **first** and **last** to store names.
- **birth and death**: Dates are stored using the date type, allowing for date-specific operations.
- **contri**s: An array of strings, useful for storing lists of contributions or tags.
- **views**: Uses `NumberLong` type for storing large numbers, such as view counts.

These examples illustrate how MongoDB's flexible schema allows for a variety of data types and structures, making it suitable for diverse applications.

## 11 / 16: MongoDB: Data Model

### MongoDB: Data Model

- **Documents can be nested**
  - Embedded sub-document
- **Denormalized data models**
  - Store related information in the same record
  - Avoids the need for a join operation
- **Normalized data models**
  - Eliminate duplication
  - Represent many-to-many relationships

```
{
  _id: <ObjectId>,
  username: "123xyz",
  contact: {
    phone: "123-456-7890",
    email: "xyz@example.com"
  },
  access: {
    level: 5,
    group: "dev"
  }
}
```

Embedded sub-document

Embedded sub-document

user document

```
{
  _id: <ObjectId>,
  username: "123xyz"
}
```

contact document

```
{
  _id: <ObjectId2>,
  user_id: <ObjectId1>,
  phone: "123-456-7890",
  email: "xyz@example.com"
}
```

access document

```
{
  _id: <ObjectId3>,
  user_id: <ObjectId1>,
  level: 5,
  group: "dev"
}
```

11 / 16

- **Documents can be nested**

- *Embedded sub-document*: In MongoDB, documents can contain other documents, known as embedded sub-documents. This allows for a hierarchical data structure within a single document, making it easier to store related data together.

- **Denormalized data models**

- *Store related information in the same record*: Denormalization involves storing related data in a single document. This approach reduces the need for complex queries and improves read performance by avoiding joins.
- *Avoids the need for a join operation*: By keeping related data together, denormalized models eliminate the need for join operations, which can be costly in terms of performance.

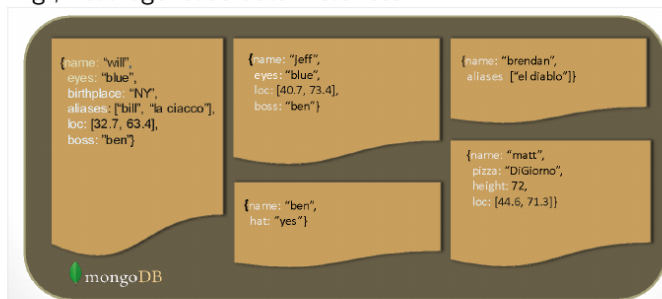
- **Normalized data models**

- *Eliminate duplication*: Normalization involves structuring data to minimize redundancy. This is achieved by separating data into different documents and linking them through references.
- *Represent many-to-many relationships*: Normalized models are useful for representing complex relationships, such as many-to-many, by using references between documents. This approach can make updates more efficient by reducing data duplication.

## 12 / 16: Schema Free

### Schema Free

- MongoDB does not need pre-defined data schema
- Every **document** in a **collection** can have different fields and values
  - No need for **NULL** values / union of fields like in relational DBs
- E.g., heterogeneous data instances



12 / 16

- **Schema Free**
  - **MongoDB does not need pre-defined data schema**
    - \* MongoDB is a NoSQL database, which means it doesn't require a fixed schema. This flexibility allows developers to store data without defining the structure beforehand, unlike traditional relational databases.
- **Every document in a collection can have different fields and values**
  - Each document in MongoDB can have its own unique set of fields and data types. This means you don't have to worry about NULL values or creating a union of fields as you would in a relational database. This flexibility is particularly useful for applications where data structures can evolve over time.
- **E.g., heterogeneous data instances**
  - The image illustrates various documents with different fields and values. For example, one document might include fields like **name**, **eyes**, and **birthplace**, while another might only have **name** and **hat**. This demonstrates MongoDB's ability to handle diverse data types and structures within the same collection, making it ideal for applications with varying data requirements.

## 13 / 16: JSON Format

**JSON Format**– **JSON = JavaScript Object Notation**

- Data is stored in field/value pairs
- A field/value pair consists of:
  - A field name (always a string)
  - Followed by a colon :
  - Followed by a typed value

```
"name": "R2-D2"
```

- Just references
- Data in documents is separated by commas ,

```
"name": "R2-D2", "race": "Droid"
```

- Curly braces {} hold documents

```
{ "name": "R2-D2", "race": "Droid", "affiliation": "rebels" }
```

- An array is stored in brackets []

```
[
  { "name": "R2-D2", "race": "Droid", "affiliation": "rebels" },
  { "name": "Yoda", "affiliation": "rebels" }
]
```

13 / 16

• **JSON Format**

- JSON stands for *JavaScript Object Notation*. It's a lightweight format used for data interchange. It's easy for humans to read and write, and easy for machines to parse and generate.

– **Data is stored in field/value pairs**

- \* Each piece of data in JSON is represented as a field/value pair. This is similar to a dictionary or a map in programming.
- \* **A field/value pair consists of:**
  - A field name, which is always a string. This acts like a label for the data.
  - A colon : separates the field name from its value.
  - A typed value follows the colon. This value can be a string, number, object, array, true, false, or null.
  - Example: "name": "R2-D2" shows a field named "name" with the value "R2-D2".

• **Data in documents is separated by commas ,**

- When you have multiple field/value pairs in a JSON document, they are separated by commas.
- Example: "name": "R2-D2", "race": "Droid" shows two pairs separated by a comma.

• **Curly braces {} hold documents**

- A JSON document is enclosed in curly braces. This is similar to how objects are defined in many programming languages.
- Example: { "name": "R2-D2", "race": "Droid", "affiliation": "rebels" } is a complete JSON object with three field/value pairs.

• **An array is stored in brackets []**

- JSON can also represent arrays, which are ordered lists of values. Arrays are enclosed in square brackets.
- Each item in the array can be a JSON object, like in the example.
- Example: The array [ { "name": "R2-D2", "race": "Droid", "affiliation": "rebels" }, { "name": "Yoda", "affiliation": "rebels" } ] contains two JSON objects.

## 14 / 16: BSON Format

### BSON Format

---

- **Binary-encoded serialization of JSON-like documents**
  - <https://bsonspec.org>
  - Similar to Protocol Buffers, but more schema-less
- **Optimized for random access**
  - Prefixes elements in a BSON document with a length field
- **MongoDB understands BSON objects, even nested ones**
  - Builds indexes and matches objects against query expressions for BSON keys

14 / 16

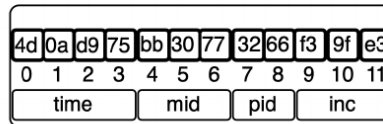
- **BSON Format**
  - *Binary-encoded serialization of JSON-like documents*
    - \* BSON stands for Binary JSON. It is a way to encode data structures that are similar to JSON but in a binary format. This makes it more efficient for computers to read and write compared to plain text JSON. You can find more details about BSON at [bsonspec.org](https://bsonspec.org).
    - \* BSON is somewhat like Protocol Buffers, which is another way to serialize data. However, BSON is more flexible because it doesn't require a predefined schema. This means you can store different types of data without having to define the structure beforehand.
- **Optimized for random access**
  - BSON is designed to allow quick access to data. Each element in a BSON document is prefixed with a length field. This means that you can easily skip over elements without having to read everything, which is useful when you only need to access specific parts of the data.
- **MongoDB understands BSON objects, even nested ones**
  - MongoDB, a popular database, uses BSON to store data. It can handle BSON objects

that contain other BSON objects inside them, known as nested objects. MongoDB can create indexes on these BSON keys, which helps in quickly finding and retrieving data based on queries. This capability makes MongoDB efficient for handling complex data structures.

## 15 / 16: ObjectId

### ObjectId

- Each JSON document contains an `_id` field of type `ObjectId`
  - Similar to the `SERIAL` constraint incrementing a numeric primary key in PostgreSQL
- An `ObjectId` is 12 bytes, composed of:
  - Timestamp
  - Client machine ID
  - Client process ID
  - 3-byte auto-incremented counter
- Each MongoDB process handles its own ID generation without collisions
  - Due to MongoDB's distributed nature
- Details here



15 / 16

- **Each JSON document contains an `_id` field of type `ObjectId`**
  - This is similar to the *SERIAL* constraint in PostgreSQL, which automatically increments a numeric primary key. In MongoDB, the `_id` field is automatically generated and serves as a unique identifier for each document.
- **An `ObjectId` is 12 bytes, composed of:**
  - **Timestamp:** The first part of the `ObjectId` is a timestamp, which ensures that the IDs are roughly ordered by creation time.
  - **Client machine ID:** This part identifies the machine where the `ObjectId` was generated, helping to ensure uniqueness across different machines.
  - **Client process ID:** This identifies the process on the machine, further ensuring that IDs are unique even if multiple processes are generating them.
  - **3-byte auto-incremented counter:** This counter is unique to each process and resets every second, ensuring that even within the same second, IDs remain unique.
- **Each MongoDB process handles its own ID generation without collisions**
  - MongoDB's distributed nature means that multiple servers can generate IDs independently without risk of duplication, thanks to the components of the `ObjectId`.
- **Details**
  - For more in-depth information, you can refer to the MongoDB documentation, which provides comprehensive details on how `ObjectId` works and its components.

## 16 / 16: Indexes

## Indexes

- **Primary index**
  - Automatically created on the `_id` field
  - B+ tree indexes
- **Secondary index**
  - Improve query performance
  - Can enforce unique values for a field
- Single field index and compound index (like SQL)
  - Order of fields in a compound index matters
- Sparse property of an index
  - Index contains entries only for documents with the indexed field
  - Ignores records without the field
- Rejects records with duplicate keys if the index is unique and sparse
- Details here

16 / 16

- **Indexes**
  - **Primary index**
    - \* This is an index that is automatically created on the `_id` field of a database. The `_id` field is a unique identifier for each document in a collection, similar to a primary key in SQL databases. The primary index uses a B+ tree structure, which is efficient for searching and retrieving data quickly.
- **Secondary index**
  - Secondary indexes are additional indexes that you can create to improve the performance of queries. They are not automatically created like the primary index. Secondary indexes can also enforce unique values for a specific field, ensuring that no two documents have the same value for that field.
- **Single field index and compound index (like SQL)**
  - A single field index is created on one field, while a compound index involves multiple fields. The order of fields in a compound index is important because it affects how queries are optimized and executed. This is similar to how indexes work in SQL databases.
- **Sparse property of an index**
  - A sparse index only includes entries for documents that have the indexed field. If a document does not have the field, it is ignored by the index. This can save space and improve performance when indexing fields that are not present in every document.
- **Rejects records with duplicate keys if the index is unique and sparse**
  - If an index is both unique and sparse, it will reject any records that would result in duplicate keys. This ensures data integrity by preventing duplicate entries for the indexed field.
- **Details here**



- For more detailed information about indexes, you can refer to the official MongoDB documentation. This resource provides comprehensive guidance on how to use and manage indexes effectively.